B.Tech CSE AIML & IoT (III YEAR - VI SEM)
(2025-2026)

DEPARTMENT OF COMPUTER ENGINEERING & APPLICATIONS

GLA UNIVERSITY MATHURA

17km Stone, NH-19, Mathura-Delhi Road P.O.
Chaumuhan, Mathura 281406 (Uttar Pradesh) India

Project Title: AI-Powered Product Condition Verification for Safer E-Commerce

Team Member 1: Kunal Singhal	UR: 2315510107
Team Member 2: Manish Bhardwaj	UR: 2315510116
Team Member 3: Kartik Agrawal	UR: 2315510095

Mentor Name: Mr. Shivanshu Upadhyay
Signature:
5/02/26

unal

Project Synopsis: <u>TrustLens AI</u> – AI-Powered Product Condition Verification for Safer E-Commerce

---

0. Cover

- **Project title:** AI-Powered Product Condition Verification for safer E-Commerce
- **Team name & ID:** Team CodeFusion
- **Institute / Course:** GLA University, Mini-Project
- **Version:** v1.0
- **Date:** 04 Feb 2026

---

1. Overview

- **Problem statement:** Current e-commerce platforms lack transparent condition verification for damaged or refurbished products. Buyers cannot judge usability from vague "damaged" labels, and sellers struggle to price items fairly, leading to low trust and high dispute rates.
- **Goal:** Build an AI-powered system that analyzes product images to detect damage, estimate severity, generate human-readable explanations, and recommend fair prices—bridging the trust gap in e-commerce.
- **Non-goals:** Image fraud detection, seller trust scoring, market-based dynamic pricing, and AI-assisted dispute resolution in MVP.
- **Value proposition:** Automated damage detection via YOLOv8, severity scoring (1-10), clear condition explanations, and rule-based pricing recommendations—all delivered within 5 seconds via a React web interface.

---

2. Scope and Control

2.1 In-scope

- Image upload (multiple product images)
- AI-based damage detection using YOLOv8
- Severity scoring (1–10) with rule-based logic
- Simple condition explanation (LLM prompt-based)
- Rule-based price recommendation
- Basic seller dashboard (upload → view result)
- Buyer result page (damage map, severity, explanation, price)
- Email-verified login/signup (optional if time permits)
- Image storage (local/S3)
- REST APIs for all core features
- Basic admin/moderation for test images
- Logging + error handling

2.2 Out-of-scope

- Market-based dynamic pricing (scraped or API-based)
- Advanced fraud detection (EXIF/ELA/reverse search)
- Seller trust scoring
- Full chatbot for interactive buyer queries
- Advanced dispute resolution
- Mobile apps (Android/iOS)
- High scalability / load balancing
- Payment systems or order workflows

2.3 Assumptions

- Product category is limited to **electronics** (mobile phones, laptops, accessories).
- All users are within project testing group (college team / mentor).
- Dataset will be collected manually + online sources (Kaggle, Google images).
- Single backend server is used (FastAPI).
- No real-time GPU server; inference may run on CPU or Colab API.

2.4 Constraints

- 12–13 week mini-project timeline.
- Limited dataset (500–1000 images).
- Limited GPU access (Colab free tier).
- Beginner/intermediate skill level in ML + full stack.
- Deployment on free-tier services only (Vercel, Render, Supabase).
- YOLOv8 inference cannot exceed heavy server load.

2.5 Dependencies

- **SMTP service** (optional) for email-based verification.
- **Object storage** for images: AWS S3 / Supabase Storage / Firebase.
- **External APIs** (optional): OpenAI/Claude for explanation module.
- **Model Training**: Google Colab for YOLOv8 training.
- **Environment**: Python 3.10+, Node.js, React, PostgreSQL.

2.6 Acceptance criteria and sign-off

- **AI Damage Detection:**
  GIVEN a user uploads a product image
  WHEN YOLO runs
  THEN bounding boxes + damage types must be returned in **≤ 5 seconds**.
- **Severity Score:**
  GIVEN YOLO outputs
  WHEN severity module runs
  THEN a severity value (1–10) is generated with correct mapping.
- **Price Recommendation:**
  GIVEN severity score and base price
  WHEN pricing engine runs
  THEN recommended price must match defined rule-based logic.
- **Explanation Module:**
  GIVEN severity + detected damage
  WHEN LLM summary is generated
  THEN user receives a **2–3 sentence** clear explanation.
- **Frontend Result Page:**
  GIVEN backend response

THEN UI displays damage map, severity, explanation, and price correctly.

Sign-off table

Stakeholder	Role	Decision area	Signature/Approval	Date
Mr. Shivanshu Upadhyay	Mentor	Scope, final acceptance	Pending	TBD
Kunal Singhal	Product Lead	Release readiness	Pending	TBD

---

3. Stakeholders and RACI

Activity	Responsible (R)	Accountable (A)	Consulted (C)	Informed (I)
Requirements	Kunal Singhal	Kunal Singhal	Mr. Shivanshu	team
AI/ML Development	Manish Bhardwaj	Manish Bhardwaj	Mr. Shivanshu	Team
Deployment	Kartik Agrawal	Kartik Agrawal	—	Team
FrontEnd	Kartik Agrawal	Kartik Agrawal	Mr. Shivanshu	Team
BackEnd	Kunal Singhal	Kunal Singhal	Mr. Shivanshu	Dept

---

4. Team and Roles

Member	Role	Responsibilities	Key skills	Availability	Contact
Kunal Singhal	Product Lead	BackEnd & AIML Developer	Python, FastApi, ML, NLP, Node.js	8 hrs/wk	kunal.singhal_cs.aiml23@gla.ac.in
Kartik Agarwal	UI/UX Developer	Frontend & Deployment	React, Tailwind CSS, JavaScript, AWS	10hrs/wk	kartik.agrawal_cs.aiml23@gla.ac.in
Manish Bhardwaj	AIML Developer	Damage Detection model, Severity Score model	ML, DL, open CV, NLP,Python	10 hrs/wk	manish.bhardwaj_cs.aiml23@gla.ac.in

---

5. Week-wise Plan and Assignments

Week	Dates	Milestones	Kunal (Lead)	Manish (Backend)	Kartik (Frontend)	Deliverables	Status
1	5-11 Feb	Project setup & planning	Setup dev environment,FastAPI skeleton	Dataset research & collection	React project init,wireframes	Project plan,dev setup	Planned

2	12-18 Feb	Database Preparation	API endpoints design,image preprocessing module	Label Dataset augmentation pipeline	UI mockups ,component structure	Labeled Dataset, API spec	Planned
3	19-25 Feb	Model training begins	FastAPI upload endpoint, file handling	YOLOv8 training on damage detection	Image upload component, file validation	Initial model v0.1	Planned
4	26 – 3 Mar	Model integration	Integrate YOLOv8, inference pipeline	Model fine-tuning, hyperparameter optimization	Results display component	Working backend API	Planned
5	4-10 Mar	Severity & pricing logic	Severity estimation algorithm	Condition explanation module	Damage visualization overlay	Severity scoring module	Planned
6	11-17 Mar	Pricing engine	Rule-based pricing engine	Testing model accuracy, edge cases	Price recommendation UI	Pricing module complete	Planned
7	18-24 Mar	Frontend-backend	API refinement,	Model performance	Full UI integration, state	End-to-end flow working	Planned

		integration	error handling	optimization	management		
8	25–31 Mar	Testing & bug fixes	Unit tests, API documentation	Model validation on test set	UI/UX improvements, responsiveness	Test results, bug list	Planned
9	2-8 Apr	Deployment preparation	Docker	Final model evaluation report	Deployment setup and configuration	Dockerized application	Planned
10	9-15Apr	User testing	Performance monitoring, logging	User feedback analysis	UI fixes based on feedback	User test report	Planned
11	16-22 Apr	Documentation & polish	API docs, README	Technical documentation	User guide, demo video	Complete documentation	Planned
12	23-28 Apr	Final presentation prep	Presentation slides	Demo preparation	Final testing, polish	Presentation, demo	Planned

---

6. Users and UX

6.1 Personas

- **New Seller Sam:** Wants to upload product images quickly and get an instant AI condition report + fair price.
- **Buyer Bella:** Wants clear, trustworthy condition explanations and honest pricing before deciding to buy.

6.2 Top user journeys

- **Seller Journey**
  Home → Upload Image → Enter Base Price → Get AI Analysis
  Damage Map + Severity + Explanation + Recommended Price shown immediately.
  KPI: Process completes in ≤ 4 steps, success rate ≥ 85%.

- **Buyer Journey**
  Home → Open Product → View AI Condition Report → View Recommended Price → Decide to buy.
  KPI: Report loads in ≤ 2 sec, clarity rating ≥ 80%.

6.3 User stories

Seller

- **Story:** As a seller, I want to upload product images and get an AI condition report in under 2 minutes so I can price and publish my listing quickly.
- **Acceptance:** GIVEN I am logged in, WHEN I upload images and submit, THEN I see the AI damage map, severity, explanation, and recommended price.

Buyer

- **Story:** As a buyer, I want a clear AI-generated condition summary so I can decide whether the product is worth buying.
- **Acceptance:** GIVEN I open a product page, WHEN the AI report loads, THEN I see damage, severity, and price recommendation within 2 seconds.

6.4 Accessibility & localization

- Basic keyboard navigation on main flows (upload, view report).
- UI follows contrast AA standards.
- Language: English only for v1.
- RTL or regional languages **not required** for mini-project.

---

7. Market and Competitors

7.1 Competitor table

Competitor	Product	Target users	Key features	Pricing	Strengths	Weaknesses	Our differentiator
OLX	C2C market place	General public	Listings, chat	Free	Large base	Ad clutter	Campus-verified users
Cashify	Electronics resale	India users	Multi-category	Free	Trusted brand	Manual condition checks only	Fully automated AI condition verification
Refurb vendors (Amazon Renewed, Flipkart Renewed)	Refurb electronics	Online buyers	Refurb checks	Free	High trust	Not available for individuals	Open to anyone; AI-powered instead of manual checks

7.2 Positioning

Unique Angle

AI-driven, transparent, and automated product condition verification for used electronics.

Measurable Differentiator

- AI report generated in ≤ 10 seconds vs. manual checks taking hours/days.

- Automated severity scoring and fraud checks not available in other platforms.
- Clear explanation + fair pricing = higher buyer trust.

---

8. Objectives and Success Metrics

1. Processing Speed

   AI damage detection + severity scoring + pricing should complete in **≤ 5 seconds** by final release.
   KPI: Median processing time (ms).

2. Explanation Quality

   At least **80% of users** (testers/mentors/classmates) should rate the AI-generated condition explanation as **"clear and helpful."**
   KPI: User clarity score.

3. Accuracy Goals

   YOLO damage detection accuracy **≥ 80%** and severity mapping accuracy **≥ 75%** on validation dataset.
   KPI: Model accuracy (%).

4. Reliability & UI Performance

   Result page should load within **≤ 2 seconds** after AI response.
   KPI: p95 page-load time.

---

9. Key Features

Feature	Description	Priority	Dependencies	Acceptance criteria
Image Upload	Upload 1–5 product images	Must	Storage	GIVEN images WHEN uploaded THEN processed without errors
AI Damage Detection	YOLOv8 detects scratches, cracks, dents	Must	YOLO model	GIVEN image WHEN analyzed THEN bounding boxes return ≤ 5 s
Severity Scoring	Converts detected damage into severity (1–10)	Must	Detection output	GIVEN detections WHEN scored THEN severity correct by logic
Price Recommendation	Rule-based price after severity discount	Must	Severity score	GIVEN base price WHEN processed THEN recommended price shown
Condition Explanation	Simple AI-generated description for buyers	Should	LLM API	GIVEN severity + damage WHEN explained THEN 2–3 sentence output
Results Dashboard	Shows damage map + severity + explanation + price	Must	Backend APIs	GIVEN API response WHEN opened THEN results visible ≤ 2 s
Profiles (Seller)	Basic seller info + past listings	Could	Auth	GIVEN profile URL WHEN opened THEN details visible
Moderation (Basic)	Hide/report invalid uploads	Could	Auth	GIVEN report WHEN submitted THEN flagged in system

10. Architecture

10.1 High-level

- **Clients:**
  React SPA (image upload, result page, dashboard)
- **Services:**
  • Auth service
  • AI Analysis service (damage detection, severity scoring, price engine)
  • Explanation service (LLM API call)
  • Results service (stores + retrieves analysis reports)
- **Data Stores:**
  • PostgreSQL (product info, results, users)
  • Object storage (images) – AWS S3 / Supabase Storage
- **Integrations:**
  • LLM API (OpenAI/Claude)
  • Optional SMTP for verification
  • Optional GPU/Colab for YOLO model inference

10.2 API spec snapshot

Endpoint	Method	Auth	Purpose	Request schema	Response schema	Codes
/api/auth/register	POST	—	Create account	email, password	201 {id}	201, 400
/api/upload	POST	JWT	Upload images	images[]	200 {fileUrls[]}	201, 400
/api/analyze	POST	JWT	Run YOLO + severity + pricing	imageUrls[], basePrice	200 {damage, severity, price, explanation}	200,400, 500
/api/report/{id}	GET	JWT	Get past analysis	id	200 {fullReport}	200, 404
/api/listings	POST	JWT	Create listing with AI results	title, images[], aiReport	201 {listingId}	201, 400, 401

/api/search	GET	__	Browse listings	q, category, page	200 {items[], total}	200

10.3 Config and secrets

- **Environment variables:** Stored in .env, never committed to Git
- **API Keys:**
  - OpenAI/Claude key
  - Database credentials
  - Storage bucket key
- **Rotation:** Rotate LLM + SMTP keys each term
- **CI/CD:** Restrict CI pipeline access to maintain secret safety

11. Data Design

11.1 Data dictionary

Entity	Field	Type	Null?	Allowed Values	Source	Notes
User	id	UUID	No	—	System	PK
User	email	String	No	RFC 5322	User	Unique
User	password_hash	String	No	—	System	Encrypted
Analysis Report	id	UUID	No	—	System	PK

Analysis Report	**image_urls**	**JSON[]**	**No**	**URLs**	**System**	**Stored in S3**
Analysis Report	**damage_ data**	**JSON**	**No**	**YOLO output**	**AI Model**	**Boxes + labels**
Analysis Report	**severity**	**Integer**	**No**	**1–10**	**AI Model**	**Scored value**
Analysis Report	**explanation**	**Text**	**No**	**—**	**LLM**	**Summary text**
Analysis Report	**recommended_price**	**Decimal(10,2)**	**No**	**≥ 0**	**Pricing Engine**	**Final price**
Listing	**id**	**UUID**	**No**	**—**	**System**	**PK**
Listing	**title**	**String(120)**	**No**	**—**	**Seller**	**Indexed**
Listing	**base_price**	**Decimal(10,2)**	**No**	**≥ 0**	**Seller**	**Input**
Listing	**report_id**	**UUID**	**No**	**—**	**System**	**FK → Analysis Report**

11.2 Schemas and migrations

- ER diagram included in Appendix.
- Database migrations versioned using Alembic (FastAPI) or Prisma (Node).
- Rollback tested on staging environment.

11.3 Privacy, retention, backup/DR

- **PII:** name, email.
- **Retention:** delete accounts & reports after 12 months of inactivity.
- **Backups:** nightly PostgreSQL backups.
- **RTO:** 4 hours (max downtime before restore).
- **RPO:** 24 hours (max data loss).

---

12. Technical Workflow Diagram

i. State Transition Diagram
– Shows transitions from *Image Upload → Analysis Running → Results Ready → Listing Created*

ii. Sequence Diagram
– Step-by-step interaction between User → Frontend → FastAPI → YOLO Model → Severity Engine → Pricing Module → LLM → Database → Frontend

iii. Use Case Diagram
– Actors: *Seller, Buyer, AI System*
– Use cases: *Upload Image, Run Analysis, View Report, Create Listing, Browse Listing*

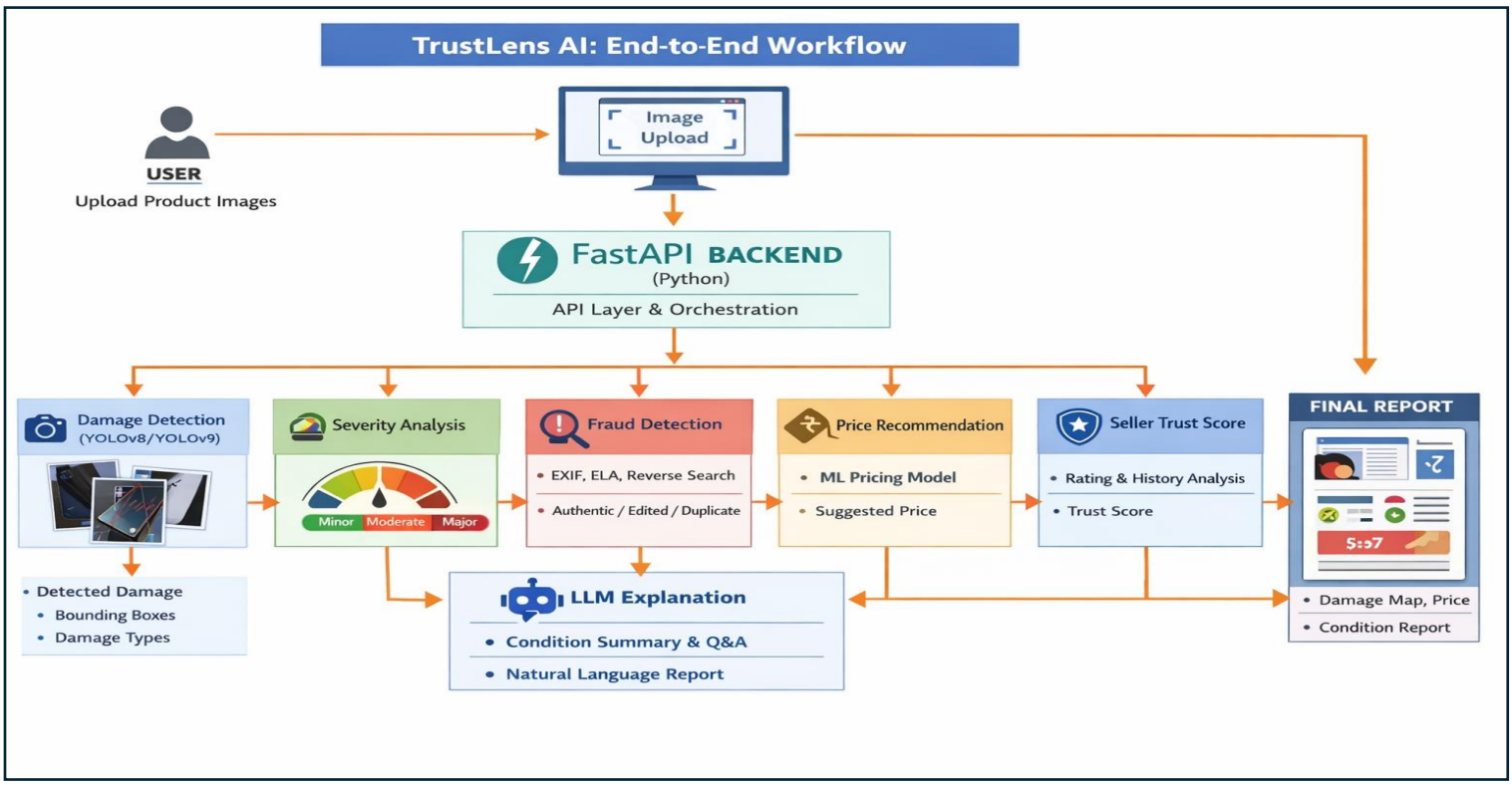
iv. Data Flow Diagram (DFD)
– Flow: *Images → Preprocessing → YOLO Detection → Severity → Pricing → Explanation → Database → UI*

v. ER Diagram
– Entities: *User, Listing, Analysis Report*
– Relationships shown using PK–FK mapping

vi. Technical Workflow Diagram
– End-to-end pipeline for AI processing and backend handling

vii. Work Architecture Diagram
– High-level architecture covering Frontend, Backend, AI Models, Storage, Database, APIs

---


TrustLens AI: End-to-End Workflow
USER
Upload Product Images
Image Upload
FastAPI BACKEND
(Python)
API Layer & Orchestration
Damage Detection
(YOLOv8/YOLOv9)
Detected Damage
Bounding Boxes
Damage Types
Severity Analysis
Minor
Moderate
Major
Fraud Detection
EXIF, ELA, Reverse Search
Authentic / Edited / Duplicate
Price Recommendation
ML Pricing Model
Suggested Price
Seller Trust Score
Rating & History Analysis
Trust Score
LLM Explanation
Condition Summary & Q&A
Natural Language Report
FINAL REPORT
Damage Map, Price
Condition Report

13. Quality: NFRs and Testing

13.1 Non-functional requirements

Metric	SLI	Target (SLO)	Measurement
Processing Speed	YOLO + severity + price time	≤ 5000 ms	APM / logs
UI Load Time	p95 load time	≤ 2000 ms	Browser profiling
Error Rate	5xx %	≤ 1%	Backend logs
Security	Open critical CVEs	0	Dependency scanner
Model Reliability	Damage detection accuracy	≥ 80%	Validation dataset

13.2 Test plan

Area	Type	Tools	Owner	Coverage Target	Exit Criteria
Backend	Unit Tests	PyTest	Team	70%	No P1/P2 defects
UI	E2E Tests	Playwright	Team	60%	Pass rate ≥ 95%
API	Integration Tests	Postman/Newman	Team	All critical scenarios	All critical pass
AI Model	Validation Tests	YOLO metrics tool	Team	≥ 80% mAP	Accuracy target met

13.3 Environments

- **Dev → Staging → Prod** pipeline
- Feature flags for risky model updates
- Staging must pass:
  - ❖ All API tests
  - ❖ All E2E flows
  - ❖ Basic model accuracy check

---

14. Security and Compliance

14.1 Threat model (STRIDE)

Asset	Threat	STRIDE	Impact	Likelihood	Mitigation	Owner
Auth tokens	Token theft	Spoofing	High	Medium	HTTPS, short-lived JWT, token rotation	Team
Uploaded images	Malware / Fake image injection	Tampering	Medium	Low	File-type validation, virus scan, size limits	Team
AI Model API	Model misuse / prompt injection	Elevation of Privilege	Medium	Low	Input sanitization, rate limiting	Team
User data	SQL injection	Tampering	High	Low	Parameterized queries, WAF	Team

14.2 AuthN/AuthZ

- **AuthN:** Email + password + optional email verification.

- **Roles:** seller, buyer, admin (role checks at API gateway).
- **JWT:** Short TTL (e.g., 15–30 min), refresh token optional.

14.3 Audit and logging

- Log:
  ✓Signups
  ✓ Logins
  ✓ Image uploads
  ✓ AI analysis requests
  ✓ Listing create/update/delete
- Retention: **90 days**
- Sensitive logs: masked (email, IP)

14.4 Compliance

- Academic project; follow institute policies.
- No third-party sharing of user data.
- AI outputs tested for fairness + transparency.
- Uploaded images stored securely (private bucket).

---

15. Delivery and Operations

15.1 Release plan

- **v1.0 Mini-Project Demo:** Scheduled for **TBD**
- Initial release includes:
  ✓ Image upload
  ✓ Damage detection
  ✓ Severity scoring
  ✓ Pricing engine
  ✓ Explanation module
- AI improvements (fraud detection, trust score) behind **feature flags**
- Optional: Dark-launch YOLO model improvements

15.2 CI/CD and rollback

- **CI Pipeline:**
  lint → unit tests → build → dockerize
- **CD Pipeline:**
  Deploy to staging on each merge; manual promotion to production
- **Rollback Strategy:**
  Revert to previous working Docker image tag

15.3 Monitoring and alerting

Metric	Threshold	Alert to	Runbook
p95 AI processing latency	> 6000 ms	Tech Lead	"AI Latency" runbook
Error rate (5xx)	> 2%	Tech Lead	"Error Spike" runbook
Model failure rate	> 5%	ML Lead	"Model Failure" runbook

15.4 Runbooks

AI Latency Runbook

- Check YOLO model server load
- Verify storage (S3) response times
- Scale backend instance if needed
- Revert recent deployment if regression suspected

Error Spike Runbook

- Inspect backend logs
- Identify failing endpoint
- Roll back to previous version
- Document issue in incident log

Model Failure Runbook

- Validate model input format
- Check version mismatch
- Switch to fallback model

- Escalate to ML lead

---

16. Risks and Mitigations

16.1 Risk heatmap

Risk	Probability	Impact	Score	Mitigation	Owner	Status
Model accuracy lower than expected	Medium	High	12	Data augmentation, hyperparameter tuning	Team (ML Lead)	Open
Slow AI processing time	Medium	Medium	9	Optimize YOLO model, use batching, reduce image size	Backend Lead	Open
High storage usage for images	Low	Medium	6	Compression, size limits, auto-delete old images	Team	Open
LLM cost overruns	Low	Medium	6	Caching responses, smaller models for MVP	Team	O

---

17. Research and Evaluation

- **Market Research:**
  Studied OLX, Cashify, and FB Marketplace to understand trust gaps and how damaged products are handled.
- **Technical Research:**
  Benchmarked YOLOv8 damage detection models, LLM summarization quality, and rule-based pricing logic.
- **Evaluation Plan:**
  • Track KPIs weekly (processing time, accuracy, clarity score).
  • After MVP release, run a user survey on 10–15 testers.
- **Limitations:**
  No payment gateway, no delivery system, single-campus test environment, basic moderation only.

---

18. Appendices

- **Glossary:**
  YOLO, mAP, LLM, JWT, KPI, SLO, p95, CVE, inference time.

- **References:**
  Ultralytics docs (YOLOv8), FastAPI docs, React docs, PostgreSQL docs, academic ML papers on defect detection.